Overview of HA cluster

links that helped me:

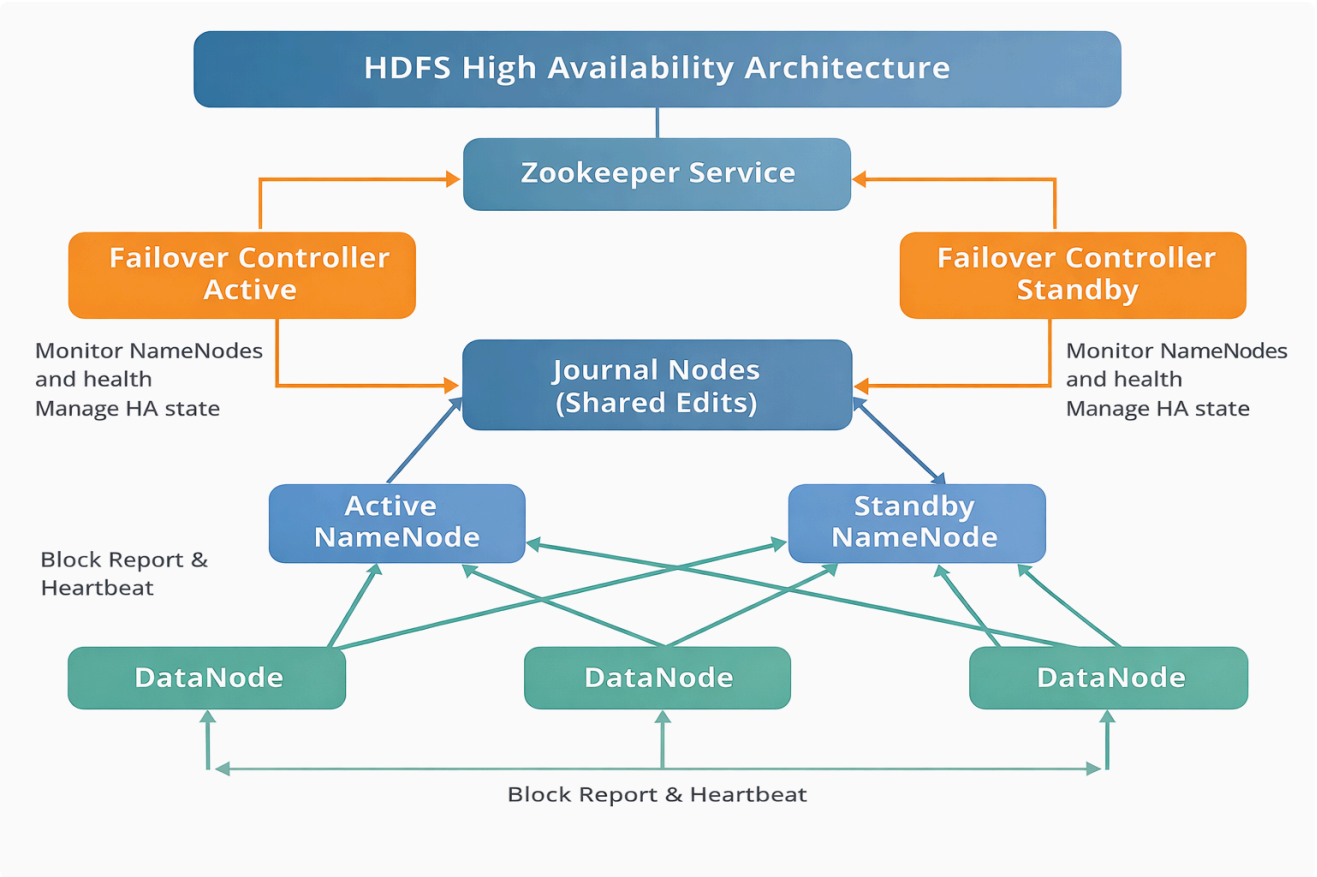
- https://www.edureka.co/blog/how-to-set-up-hadoop-cluster-with-hdfs-high-availability/
- https://phoenixnap.com/kb/install-hadoop-ubuntu
- https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/SingleCluster.html#Fully-Distributed_Operation
- https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/ClusterSetup.html
- https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-common/SingleCluster.html
- https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/ResourceManagerHA.html
- https://data-flair.training/blogs/hadoop-namenode-automatic-failover/
- https://sonu-tyagi.medium.com/different-configuration-xmls-in-hadoop-an-in-depth-look-65b4adc60282
- https://www.tutorialspoint.com/zookeeper/zookeeper_installation.htm

HDFS HA Architecture:

Let us understand that how HDFS HA Architecture solved this critical problem of NameNode availability:

The HA architecture solved this problem of NameNode availability by allowing us to have two NameNodes in an active/passive configuration. So, we have two running NameNodes at the same time in a High Availability cluster:

- Active NameNode
- Standby/Passive NameNode.



If one NameNode goes down, the other NameNode can take over the responsibility and therefore, reduce the cluster down time. The standby NameNode serves the purpose of a backup NameNode (unlike the Secondary NameNode) which incorporate failover capabilities to the Hadoop cluster.

in order to achieve that:

1. Active and Standby NameNode should always be in sync with each other, i.e. They should have the same metadata
2. There should be only one active NameNode at a time because two active NameNode will lead to corruption of the data. _This kind of scenario is termed as a split-brain scenario
3. To avoid such scenarios fencing is done. Fencing is a process of ensuring that only one NameNode remains active at a particular time.

Fencing = Forcefully killing the old Active NameNode before promoting the new one.
"Before NameNode 2 becomes Active — KILL NameNode 1 first, no matter what."

there are 3 fencing methods we can use in hdfs-site.xml configuration in hadoop:

- sshfence # most common
- shell(path/to/kill/process) # fencing through script
- shell(bin/true) # fake fencing method

method 1&2 need to be able to ssh to other NameNode to kill it

---

1. Using Quorum Journal Nodes:

The standby NameNode and the active NameNode keep in sync with each other through a separate group of nodes or daemons - called JournalNodes. The JournalNodes follows the ring topology where the nodes are connected to each other to form a ring

steps:

- The JournalNode serves the request coming to it and copies the information into other nodes in the ring. This provides fault tolerance in case of JournalNode failure as it works in concept of majority (at least 3 JN to tolerate one fault of them).
- The active NameNode is responsible for updating the EditLogs (metadata information) present in the JournalNodes.
- The StandbyNode reads the changes made to the EditLogs in the JournalNode and applies it to its own namespace in a constant manner.

- The IP Addresses of both the NameNodes are available to all the DataNodes and they send their heartbeats and block location information to both the NameNode. This provides a fast failover (less down time) as the StandbyNode has an updated information about the block location in the cluster.

Fencing of NameNode:

The JournalNodes performs this fencing by allowing only one NameNode to be the writer at a time.

During failover of active NameNode:

- The Standby NameNode takes over the responsibility of writing to the JournalNodes and forbid any other NameNode to remain active.
- Finally, the new Active NameNode can perform its activities safely.
- The administrator must configure at least one fencing method to avoid a split-brain scenario.
- The system may employ a range of fencing mechanisms. It may include killing of the NameNode's process and revoking its access to the shared storage directory.
- As a last resort, we can fence the previously active NameNode with a technique known as STONITH, or "shoot the other node in the head". STONITH uses a specialized power distribution unit to forcibly power down the NameNode machine.

Automatic Failover:

Failover is a procedure by which a system automatically transfers control to secondary system when it detects a fault or failure. There are two types of failover:

Graceful Failover: In this case, we manually initiate the failover for routine maintenance.
Automatic Failover: In this case, the failover is initiated automatically in case of NameNode failure (unplanned event).

Apache Zookeeper is a service that provides the automatic failover capability in HDFS High Availabilty cluster. It maintains small amounts of coordination data, informs clients of changes in that data, and monitors clients for failures.

> ZooKeeper is like a "Referee + Watchdog" for your Hadoop cluster:
>
> 1. 📋 Stores coordination data → "Who is the Active NameNode?"
> 2. 📢 Informs clients of changes → "NN1 just died! NN2 is now Active!"
> 3. 👁️ Monitors clients → "Is NN1 still alive? Still alive? ... DEAD!"

Zookeeper maintains a session with the NameNodes. In case of failure, the session will expire and the Zookeeper will inform other NameNodes to initiate the failover process. In case of NameNode failure, other passive NameNode can take a lock in Zookeeper stating that it wants to become the next Active NameNode.

The ZookeerFailoverController (ZKFC) is a Zookeeper client that also monitors and manages the NameNode status. Each of the NameNode runs a ZKFC also. ZKFC is responsible for monitoring the health of the NameNodes periodically.

for setup of my cluster:
[Setup for HA HDFS Cluster]

Architecture of my cluster

```
PS H:\omar\ITI - Data Management\hadoop-project> docker exec -it master1 bash
root@master1:/# jps
24 Jps
root@master1:/# ./start_cluster.sh
=== Starting ZooKeeper ===
ZooKeeper JMX enabled by default
Using config: /apache-zookeeper-3.8.6-bin/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /apache-zookeeper-3.8.6-bin/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /apache-zookeeper-3.8.6-bin/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
=== Starting JournalNodes ===
=== Starting NameNodes ===
=== Starting ZKFC ===
=== Starting DataNodes ===
=== Starting YARN ===
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
=== Cluster Status ===
--- master1 ---
291 NameNode
566 ResourceManager
440 DFSZKFailoverController
843 Jps
187 JournalNode
76 QuorumPeerMain
--- master2 ---
656 Jps
272 NameNode
563 ResourceManager
441 DFSZKFailoverController
169 JournalNode
43 QuorumPeerMain
--- worker1 ---
162 JournalNode
611 Jps
298 DataNode
43 QuorumPeerMain
461 NodeManager
--- worker2 ---
200 NodeManager
350 Jps
63 DataNode
--- worker3 ---
200 NodeManager
317 Jps
62 DataNode
=== HA Status ===
master1:8020                                        active
master2:8020                                        standby
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
master1:8033                                        active
master2:8033                                        standby
root@master1:/#
```

Advantages

NameNode High Availability
ResourceManager High Availability
ZooKeeper Quorum (3 Nodes) ==> fault tolerance
JournalNode Quorum (3 Nodes) ==> fault tolerance
JN with NN ==> less network barrier
RM with NN ==>
NM with DN ==> Data locality where processing (NM) is with data blocks (DN)

Disadvantage

Worker1 is very loaded
Mixed roles (Nothing dedicated)

2 Data nodes and separate ZK VS 3 Data nodes where on of them has ZK?

Option1:

things are separated and roles are dedicated and clear with less load on important process like ZK on big workloads on datanodes

BUT,

- architecture design will fail on applying default configurations like data replication=3
- few data blocks to store data

Option2

- architecture design comply with default configurations like data replication=3
- more data blocks to store data

BUT,

- ZK is lightweight process but can be impacted on Huge workloads on datanodes (DN take most of the resources)
- roles are not clear